

Operator: `Linear` (Fully Connected)

Phase 1.5 nấc 2 → đầu vào cho Phase 2A (viết operator) và Phase 2D (golden model C).

1. Vai trò

Mỗi đầu ra nhìn **toàn bộ** đầu vào. Không có cửa sổ trượt, không dùng lại trọng số ở đâu cả: mỗi cặp (đầu vào, đầu ra) có một trọng số riêng.

Đó vừa là sức mạnh — nó tổng hợp được thông tin từ mọi vị trí trong ảnh — vừa là lý do nó đắt.

`Conv2d` dùng 1 bộ kernel cho cả ảnh; `Linear` phải có một số riêng cho từng liên kết.

Trong ONNX nó hiện ra là node `Gemm` (General Matrix Multiply).

2. Công thức

```
y[o] = bias[o] + Σ (i = 0 .. IN-1) W[o][i] × x[i]
```

```
params = in_features × out_features + out_features  
        └── bias
```

```
MAC      = in_features × out_features
```

Chú ý: `params` và `MAC` gần như **bằng nhau**, chỉ lệch đúng phần bias. Điều này không đúng với conv, và mục 5 sẽ cho thấy vì sao nó quan trọng.

Công thức `params` **có** `in_features`, mà `in_features` đến từ shape sau `flatten` — tức là phụ thuộc kích thước ảnh. Đây chính là điểm khác căn bản với `Conv2d`, và là câu trả lời cho "vì sao `fc1` của `DigitCNN` chiếm 97% tham số".

3. Mã giả

```
linear(input[IN], weight[OUT][IN], bias[OUT], output[OUT]):  
  
    for o = 0 .. OUT-1:  
  
        acc = bias[o]                                # khởi tạo bằng bias, KHÔNG phải 0  
  
        for i = 0 .. IN-1:  
            acc += weight[o][i] * input[i]  
  
        output[o] = acc
```

Hai vòng lặp. Đơn giản hơn `Conv2d` (6 vòng) rất nhiều — nhưng đơn giản không có nghĩa là rẻ trên phần cứng.

4. Áp dụng vào `SmallCNN`

	Giá trị
<code>in_features</code>	144 (= 16 × 3 × 3)
<code>out_features</code>	10 (10 chữ số)
Params	144 × 10 + 10 = 1.450
MAC	144 × 10 = 1.440
Tỉ lệ trong tổng 5.018 params	28,9%

So với `DigitCNN`:

Model	Lớp FC	in → out	Params	% toàn mạng
DigitCNN	<code>fc1</code>	1568 → 128	200.704	97,06%
<code>SmallCNN</code>	<code>fc</code>	144 → 10	1.450	28,9%

Giảm 138 lần, và toàn bộ đến từ việc ép spatial xuống trước khi `flatten` cộng với bỏ hẳn lớp FC ẩn.

5. Mức FPGA-friendly

⚠ Phép toán thì tốt, kiểu truy cập bộ nhớ thì tệ.

Bản thân phép tính vẫn là MAC, giống conv, không có gì khó. Vấn đề nằm ở **mỗi trọng số được dùng lại bao nhiêu lần**:

Lớp	Params	MAC	Mỗi trọng số dùng lại
<code>conv1</code>	72	56.448	784 lần
<code>conv2</code>	1.152	225.792	196 lần
<code>conv3</code>	2.304	112.896	49 lần
<code>fc</code>	1.440	1.440	1 lần

Conv đọc một trọng số vào thanh ghi rồi dùng nó hàng trăm lần — chi phí đọc bộ nhớ được chia đều ra. FC đọc một trọng số, nhân **đúng một lần**, rồi vứt đi.

Hệ quả: FC bị giới hạn bởi **băng thông bộ nhớ**, không phải bởi số bộ nhân. Thêm DSP slice vào cũng không làm nó nhanh hơn, vì DSP phải ngồi chờ dữ liệu. Đây là lý do thật sự khiến `fc1` của `DigitCNN` (200.704 trọng số, mỗi cái dùng 1 lần) là thảm họa cho phần cứng — không chỉ vì nó to, mà vì nó gốn băng thông mà không sinh ra phép tính tương xứng.

Đây cũng là lý do lượng tử hoá có lợi cho FC nhiều hơn cho conv: giảm 4 lần số byte phải đọc nghĩa là giảm thẳng 4 lần thời gian chạy của lớp bị nghẽn băng thông.

6. Chú ý khi port sang C

Bẫy	Chi tiết
Thứ tự chiều trọng số	PyTorch lưu <code>weight[out_features][in_features] → W[o][i]</code> . Keras Dense lưu <code>[in][out]</code> — chuyển vị . Đọc file Keras bằng chỉ số PyTorch thì ra ma trận lộn, không crash, kết quả sai hoàn toàn
Khởi tạo <code>acc</code>	Bằng <code>bias[o]</code> , không phải 0. Giống hệt bẫy ở <code>Conv2d</code>
Tràn accumulator ở int8	144 tích của <code>int8 × int8</code> có thể lên tới $144 × 127 × 127 = \mathbf{2.322.576}$. <code>int16</code> chỉ chứa tới 32.767 → tràn . Accumulator bắt buộc là <code>int32</code> . Đây là lỗi thật, không phải lý thuyết
Bias ở fixed-point	Bias nằm cùng thang với accumulator (tích của 2 số), không cùng thang với trọng số. Lượng tử hoá bias như trọng số là sai thang — thực tế phần cứng để bias ở <code>int32</code>
Không có softmax	<code>fc</code> là lớp cuối, xuất logit thô. Đừng thêm softmax — xem <code>argmax.md</code>

7. Tự kiểm tra

Làm trên giấy:

- Muốn `fc` có **không quá 1.000** tham số (vẫn ra 10 lớp), `in_features` tối đa là bao nhiêu? Với 16 kênh thì spatial phải xuống còn bao nhiêu × bao nhiêu?
- Với `int8`, tính giá trị tuyệt đối lớn nhất mà accumulator của `fc` có thể đạt. Cần tối thiểu bao nhiêu bit?
- `fc1` của `DigitCNN` ($1568 → 128$): mỗi trọng số được dùng lại bao nhiêu lần? So với `conv2` của `SmallCNN`, chênh bao nhiêu lần?
- Nếu thay `fc` bằng một `Conv2d(16 → 10, kernel 3, padding 0)` trên feature map $3 × 3$, số params là bao nhiêu? Kết quả có khác `fc` không — vì sao?

Đáp án

Câu 1 — `in_features` tối đa để `fc` không quá 1.000 tham số

```
params = in_features × 10 + 10 ≤ 1.000
in_features × 10          ≤    990
in_features                ≤     99
```

Tối đa 99. Kiểm: $99 × 10 + 10 = 1.000$ ✓, $100 × 10 + 10 = 1.010$ ✗.

Với 16 kênh: $16 × H × W ≤ 99 → H × W ≤ 6,19$.

Spatial	in_features	fc params	Đạt?
3×3 (hiện tại)	144	1.450	✗
2×2	64	650	✓

Phải xuống 2×2 — tức cần thêm một lần giảm kích thước nữa so với kiến trúc hiện tại.

Câu 2 — Accumulator int8 của fc cần bao nhiêu bit

```

mỗi tích tối đa: 127 × 127 = 16.129
cộng dồn 144 lần: 144 × 16.129 = 2.322.576

```

```

221 = 2.097.152 < 2.322.576 → 21 bit chưa đủ
222 = 4.194.304 > 2.322.576 → 22 bit đủ phần giá trị
+ 1 bit dấu (accumulator có thể âm)
→ 23 bit

```

Cần tối thiểu 23 bit. Không có kiểu 23-bit nên trong C phải dùng `**int32_t**`. `int16_t` (trần 32.767) tràn từ rất sớm — chỉ cần 3 tích cùng dấu là vượt.

Trên FPGA thì khác: ở đó **chọn đúng 23 bit được**, không phải làm tròn lên 32. Tiết kiệm 9 bit trên mỗi thanh ghi accumulator.

Câu 3 — Mỗi trọng số được dùng lại bao nhiêu lần

`fc1` là lớp `Linear` — không có chiều không gian để trượt, nên **mỗi trọng số dùng đúng 1 lần**.

```

fc1 (DigitCNN) : 1 lần
conv2 (SmallCNN) : H_out × W_out = 14 × 14 = 196 lần

```

Chênh 196 lần. Đây chính là lý do `fc1` nghẽn băng thông: 200.704 trọng số, mỗi cái đọc từ bộ nhớ một lần rồi bỏ, sinh ra đúng một phép nhân. Đọc rất nhiều byte để làm rất ít việc. Thêm DSP không cứu được, phải tăng băng thông bộ nhớ.

Câu 4 — Thay fc bằng Conv2d(16 → 10, kernel 3, padding 0)

```

params = k×k×in_ch×out_ch + out_ch = 3×3×16×10 + 10 = 1.440 + 10 = 1.450

```

****Bảng đúng fc**** ($144 \times 10 + 10 = 1.450$). MAC cũng bằng nhau (1.440). Lý do:

```

H_out = floor((3 + 2×0 - 3) / 1) + 1 = 1

```

Với `kernel = 3` trên feature map 3×3 và `padding = 0`, cửa sổ chỉ đặt được **đúng một vị trí**, phủ trọn feature map, không trượt đi đâu. Tích chập khi cửa sổ không trượt chính là phép nhân ma trận của `Linear`. Đầu ra 10×1×1 thay vì 10, nhưng cùng 10 con số.

Đổi qua lại không cần train lại: `fc.weight` có shape `[10, 144]`, conv cần `[10, 16, 3, 3]`. Mà $144 = 16 \times 3 \times 3$ và `flatten` đánh chỉ số theo NCHW ($i = c \cdot 9 + h \cdot 3 + w$) — đúng bằng thứ tự chiều

của conv. Nên chỉ cần `conv.weight = fc.weight.view(10, 16, 3, 3)` và copy nguyên bias.

Có khác không? Có, nhưng chỉ ở mức làm tròn. Đo bằng `study/kiem_linear_vs_conv.py` trên 10.000 ảnh test:

Đo được	Giá trị
Sai lệch tuyệt đối lớn nhất	2,174e-04
Logit lớn nhất	60,356
Sai lệch tương đối	** 3,603e-06 ** = 30 lần <code>eps</code> của float32
Ảnh dự đoán khác nhau	0 / 10.000

Tương đương về mặt **toán học**, nhưng **không bit-exact**: PyTorch chạy `Linear` bằng phép nhân ma trận còn `Conv2d` qua `im2col`, cộng dồn 144 số hạng theo hai thứ tự khác nhau thì float32 làm tròn khác nhau. Chữ "giống hệt từng bit" để dành cho hai chỗ thật sự bit-exact: bỏ softmax và đổi thứ tự `relu` ↔ `maxpool`.

****Kết luận: giữ `Linear`.** Không tiết kiệm được tham số hay phép tính nào, trong khi golden model C sẽ phải viết 6 vòng lặp thay vì 2. Phương án conv chỉ có giá trị ở giai đoạn thiết kế RTL, nếu muốn một khối phần cứng duy nhất xử lý cả conv lẫn fc.